

Csharp_week4_Day_3_Hands_on_Gurunath_Rageni

Level-1 Problem 1: Student Grade Calculator

Scenario:

A school wants to calculate the average marks of a student using a class-based approach.

Requirements:

1. Create a class Student.
2. Create method CalculateAverage(int m1, int m2, int m3).
3. Return the average marks.
4. Display grade based on average.

Technical Constraints:

1. Use return type double for average.
2. Avoid hard-coded values.

Expectations:

Clear separation of logic inside methods.

Learning Outcome:

Learn method creation, return values, and basic OOP concepts.

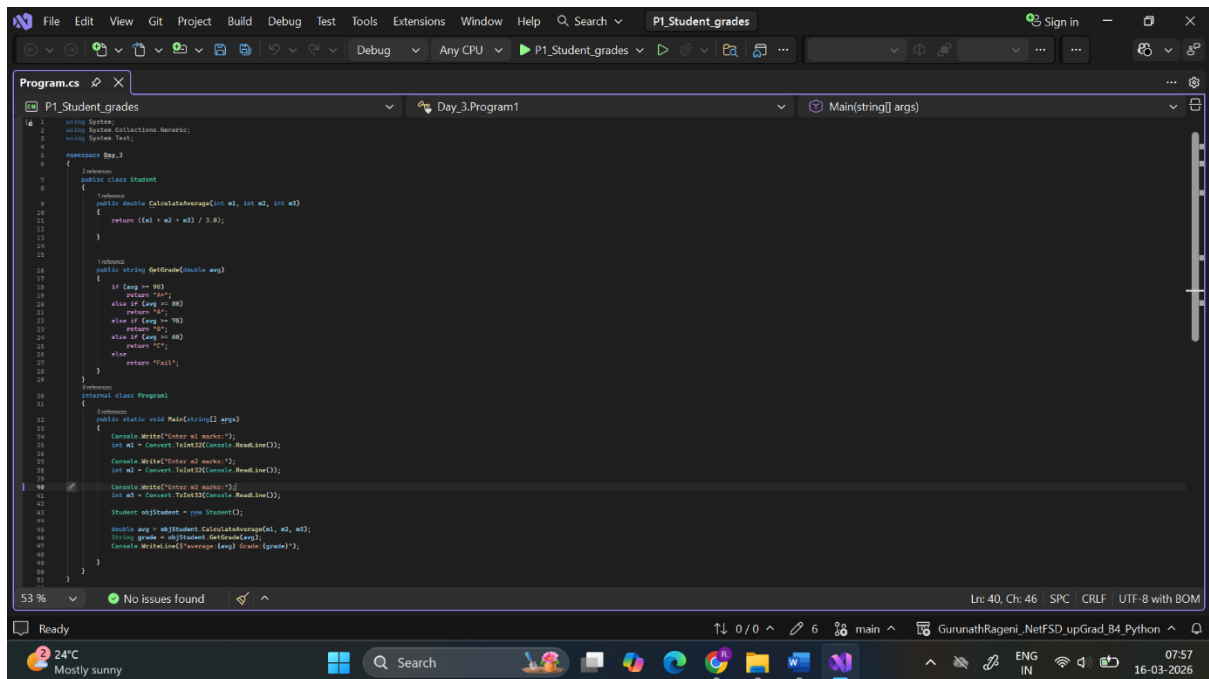
Sample Input:

80 70 90

Sample Output:

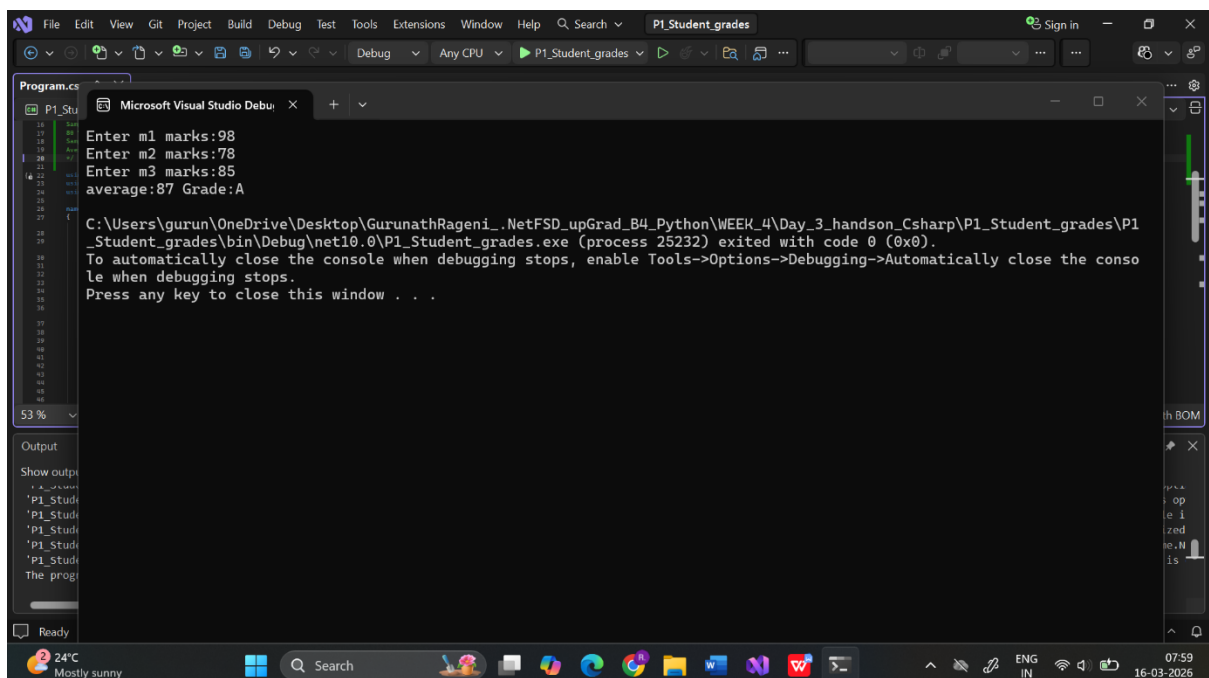
Average = 80, Grade = A

Code screenshot:



```
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace Day_3
6 {
7     [Serializable]
8     public class Student
9     {
10         public double CalculateAverage(int m1, int m2, int m3)
11         {
12             return ((m1 + m2 + m3) / 3.0);
13         }
14     }
15
16     public string GetGrade(double avg)
17     {
18         if (avg >= 90)
19             return "A+";
20         else if (avg >= 80)
21             return "A";
22         else if (avg >= 70)
23             return "B+";
24         else if (avg >= 60)
25             return "B";
26         else if (avg >= 50)
27             return "C";
28         else
29             return "Fail";
30     }
31
32     internal class Program
33     {
34         public static void Main(string[] args)
35         {
36             Console.WriteLine("Enter m1 marks:");
37             int m1 = Convert.ToInt32(Console.ReadLine());
38
39             Console.WriteLine("Enter m2 marks:");
40             int m2 = Convert.ToInt32(Console.ReadLine());
41
42             Console.WriteLine("Enter m3 marks:");
43             int m3 = Convert.ToInt32(Console.ReadLine());
44
45             Student objStudent = new Student();
46
47             double avg = objStudent.CalculateAverage(m1, m2, m3);
48             String grade = objStudent.GetGrade(avg);
49             Console.WriteLine($"Average: {avg} Grade: {grade}");
50         }
51     }
52 }
```

Output:



```
14
15
16 Enter m1 marks:98
17
18 Enter m2 marks:78
19
20 Enter m3 marks:85
21
22 average:87 Grade:A
23
24 C:\Users\gurun\OneDrive\Desktop\GurunathRageni_.NetFSD_upGrad_B4_Python\WEEK_4\Day_3_handson_Csharp\P1_Student_grades\P1_Student_grades\bin\Debug\net10.0\P1_Student_grades.exe (process 25232) exited with code 0 (0x0).
25 To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
26 Press any key to close this window . . .
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
```

Level-2 Problem 1: Bank Account with Encapsulation

Scenario:

A bank wants to manage customer accounts securely using encapsulation.

Requirements:

1. Create class BankAccount.
2. Private field: balance.
3. Public methods: Deposit(double amount), Withdraw(double amount).
4. Method GetBalance() to return balance.
5. Prevent withdrawal if insufficient balance.

Technical Constraints:

1. Balance must be private.
2. Access balance only through public methods.
3. Use appropriate return types.

Expectations:

Proper use of encapsulation and object-oriented principles.

Learning Outcome:

Understand encapsulation, access modifiers, and secure data handling.

Sample Input:

Deposit 1000, Withdraw 300

Sample Output:

Current Balance = 700

Code screenshot:

```
Enter 1 for Deposit
Enter 2 for Withdraw
Enter 3 for Check Balance
Enter 4 for Exit
Enter Your Option:1
Enter amount for Deposit: 1000000
Remaining Balance: 1000000
Enter 1 for Deposit
Enter 2 for Withdraw
Enter 3 for Check Balance
Enter 4 for Exit
Enter Your Option:4
You have successfully exit
```

Level-2 Problem 2: Bank Account Management System

Scenario:

A bank wants to develop a simple console-based application to manage customer bank accounts. The system should protect account balance information and allow controlled access using properties.

Requirements:

1. Create a BankAccount class with private fields for account number and balance.
2. Use properties to allow controlled access to account number and balance.
3. Implement Deposit and Withdraw methods with proper validation.
4. Prevent withdrawal if balance is insufficient.

Technical Constraints:

- Use private fields with public properties.
- Apply encapsulation and data hiding.
- No direct access to balance field from outside the class.

Expectations:

- Demonstrate correct use of access modifiers.
- Validate negative deposit or withdrawal amounts.
- Display updated balance after each transaction.

Learning Outcome:

- Understand encapsulation using properties.
- Apply data hiding effectively.
- Implement validation logic inside class methods.

Sample Input:

Deposit = 5000, Withdraw = 2000

Sample Output:

Current Balance = 3000

Code screenshot:

```
C# P3_Banksystem_Encapsulation_datahiding
25 using System;
26 namespace BankApp
27 {
28     2 references
29     class BankAccount
30     {
31         // Private fields (data hiding)
32         private int accountNumber;
33         private double balance;
34
35         // Property for Account Number
36         1 reference
37         public int AccountNumber
38         {
39             get { return accountNumber; }
40             set { accountNumber = value; }
41         }
42
43         // Property for Balance (read-only outside)
44         1 reference
45         public double Balance
46         {
47             get { return balance; }
48         }
49
50         1 reference
51         public void Deposit(double amount)
52         {
53             if (amount <= 0)
54             {
55                 Console.WriteLine("Invalid deposit amount");
56             }
57             else
58             {
59                 balance += amount;
60                 Console.WriteLine("Amount Deposited: " + amount);
61                 Console.WriteLine("Current Balance = " + balance);
62             }
63         }
64
65         // Withdraw Method
66         1 reference
67         public void Withdraw(double amount)
68         {
69             if (amount <= 0)
70             {
71                 Console.WriteLine("Invalid withdrawal amount");
72             }
73             else if (amount > balance)
74             {
75                 Console.WriteLine("Insufficient Balance");
76             }
77             else
78             {
79                 balance -= amount;
80                 Console.WriteLine("Amount Withdrawn: " + amount);
81                 Console.WriteLine("Current Balance = " + balance);
82             }
83         }
84     }
85
86     0 references
87     class Program
88     {
89         0 references
90         static void Main(string[] args)
91         {
92             BankAccount account = new BankAccount();
93             account.AccountNumber = 101;
94
95             Console.Write("Enter Deposit Amount: ");
96             double deposit = Convert.ToDouble(Console.ReadLine());
97             account.Deposit(deposit);
98
99             Console.Write("Enter Withdraw Amount: ");
100            double withdraw = Convert.ToDouble(Console.ReadLine());
101            account.Withdraw(withdraw);
102
103            Console.WriteLine("Final Balance = " + account.Balance);
104        }
105    }
106 }
```

36 % No issues found

Output Screenshot:

```
Enter Deposit Amount: 5000000
Amount Deposited: 500000
Current Balance = 500000
Enter Withdraw Amount: 10000
Amount Withdrawn: 10000
Current Balance = 490000
Final Balance = 490000
```

Level-2 Problem 3: Employee Salary Calculator

Scenario:

A company wants to calculate employee salaries using inheritance. All employees have a base salary, but different roles calculate bonuses differently.

Requirements:

1. Create a base class Employee with Name and BaseSalary properties.
2. Create derived classes Manager and Developer.
3. Override a virtual method CalculateSalary().
4. Manager gets 20% bonus, Developer gets 10% bonus.

Technical Constraints:

- Use inheritance and method overriding.
- Apply polymorphism using base class reference.
- Use properties for salary details.

Expectations:

- Demonstrate runtime polymorphism.
- Avoid code duplication.
- Display final calculated salary.

Learning Outcome:

- Understand inheritance hierarchy.

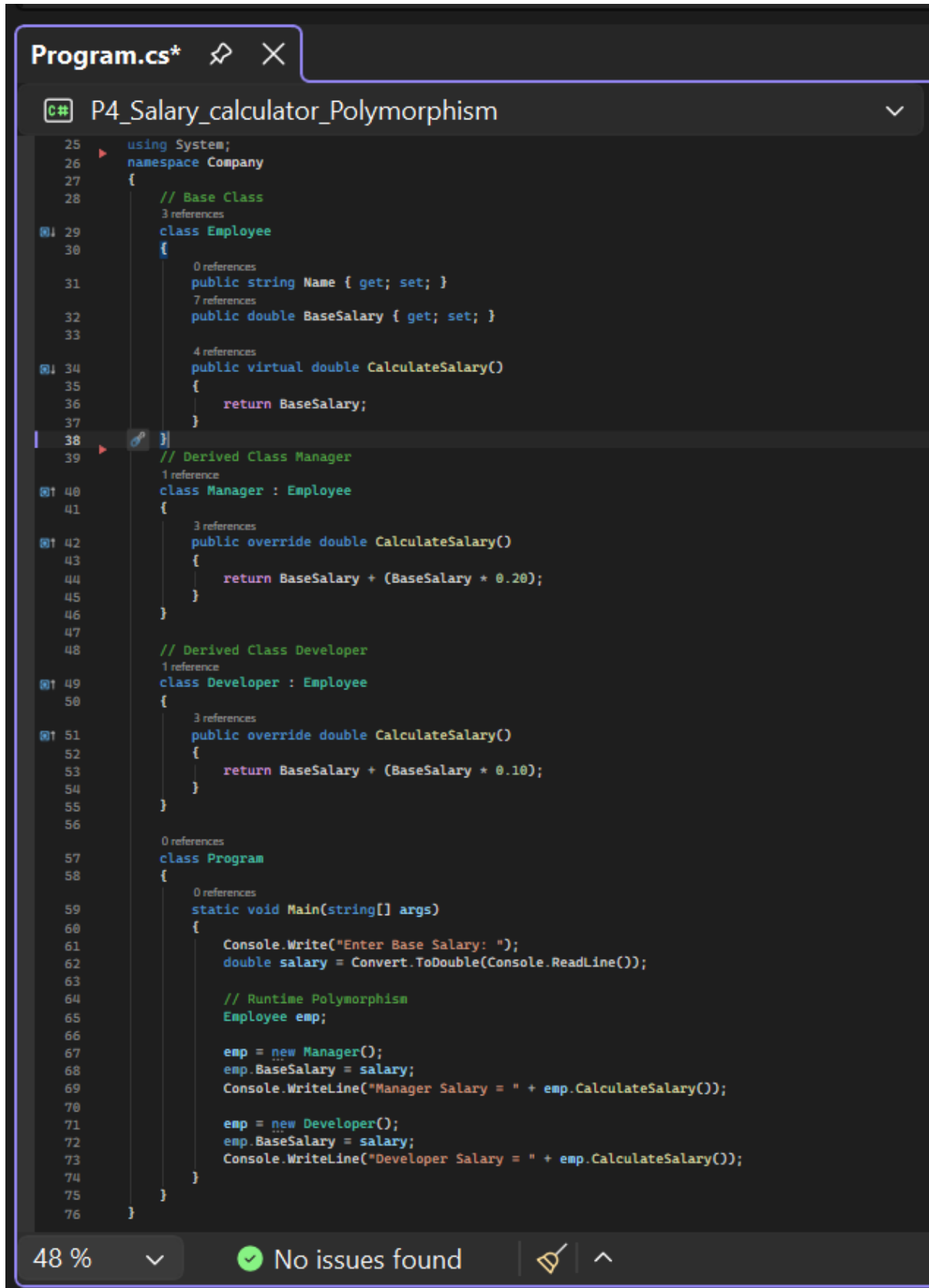
Sample Input:

BaseSalary = 50000

Sample Output:

Manager Salary = 60000, Developer Salary = 55000

Code Screenshot:



```
Program.cs* [Pin] [Close]
C# P4_Salary_calculator_Polymorphism
25 using System;
26 namespace Company
27 {
28     // Base Class
29     class Employee
30     {
31         public string Name { get; set; }
32         public double BaseSalary { get; set; }
33
34         public virtual double CalculateSalary()
35         {
36             return BaseSalary;
37         }
38     }
39     // Derived Class Manager
40     class Manager : Employee
41     {
42         public override double CalculateSalary()
43         {
44             return BaseSalary + (BaseSalary * 0.20);
45         }
46     }
47
48     // Derived Class Developer
49     class Developer : Employee
50     {
51         public override double CalculateSalary()
52         {
53             return BaseSalary + (BaseSalary * 0.10);
54         }
55     }
56
57     class Program
58     {
59         static void Main(string[] args)
60         {
61             Console.WriteLine("Enter Base Salary: ");
62             double salary = Convert.ToDouble(Console.ReadLine());
63
64             // Runtime Polymorphism
65             Employee emp;
66
67             emp = new Manager();
68             emp.BaseSalary = salary;
69             Console.WriteLine("Manager Salary = " + emp.CalculateSalary());
70
71             emp = new Developer();
72             emp.BaseSalary = salary;
73             Console.WriteLine("Developer Salary = " + emp.CalculateSalary());
74         }
75     }
76 }
```

48 % [Dropdown] [Checkmark] No issues found [Find] [Up Arrow]

Output Screenshot:

```
Enter Base Salary: 50000
Manager Salary = 60000
Developer Salary = 55000
```

Level-2 Problem 4: Online Shopping Cart System

Scenario:

An e-commerce platform needs a flexible cart system where different product types calculate discounts differently.

Requirements:

1. Create a base class Product with properties Name and Price.
2. Create derived classes Electronics and Clothing.
3. Implement a virtual method CalculateDiscount().
4. Electronics get 5% discount, Clothing gets 15% discount.
5. Use encapsulation to protect price updates.

Technical Constraints:

- Use private fields with public properties.
- Apply inheritance and method overriding.
- Prevent negative price assignment.

Expectations:

- Demonstrate polymorphic behavior in cart processing.
- Apply data validation inside properties.
- Calculate and display final price after discount.

Learning Outcome:

- Combine encapsulation and polymorphism.
- Design extensible product hierarchy.
- Implement business logic in overridden methods.

Sample Input: Electronics Price = 20000

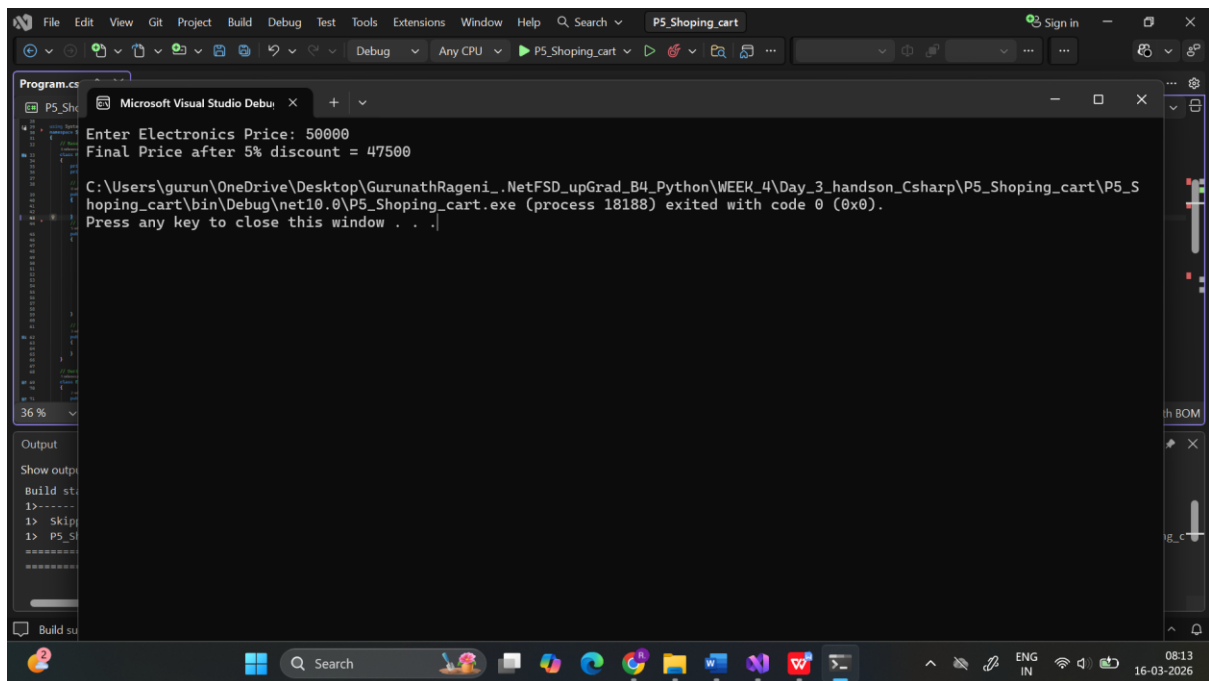
Sample Output: Final Price after 5% discount = 19000

Code Screenshot:

```
Program.cs*  
C# P5_Shopping_cart  
28
29 using System;
30 namespace ShoppingCart
31 {
32     // Base Class
33     3 references
34     class Product
35     {
36         private string name;
37         private double price;
38
39         // Property for Name
40         0 references
41         public string Name
42         {
43             get { return name; }
44             set { name = value; }
45         }
46
47         // Property for Price with validation
48         5 references
49         public double Price
50         {
51             get { return price; }
52             set
53             {
54                 if (value < 0)
55                 {
56                     Console.WriteLine("Price cannot be negative");
57                 }
58                 else
59                 {
60                     price = value;
61                 }
62             }
63         }
64
65         // Virtual Method
66         3 references
67         public virtual double CalculateDiscount()
68         {
69             return price;
70         }
71     }
72
73     // Derived Class Electronics
74     1 reference
75     class Electronics : Product
76     {
77         2 references
78         public override double CalculateDiscount()
79         {
80             return Price - (Price * 0.05);
81         }
82     }
83
84     // Derived Class Clothing
85     0 references
86     class Clothing : Product
87     {
88         2 references
89         public override double CalculateDiscount()
90         {
91             return Price - (Price * 0.15);
92         }
93     }
94
95     0 references
96     class Program
97     {
98         0 references
99         static void Main(string[] args)
100         {
101             Console.Write("Enter Electronics Price: ");
102             double price = Convert.ToDouble(Console.ReadLine());
103
104             // Polymorphism
105             Product product = new Electronics();
106             product.Price = price;
107
108             Console.WriteLine("Final Price after 5% discount = " + product.CalculateDiscount());
109         }
110     }
111 }
```

36 % 0 1

Output Screenshot:



```
Microsoft Visual Studio Debug Console
Enter Electronics Price: 50000
Final Price after 5% discount = 47500
C:\Users\gurun\OneDrive\Desktop\GurunathRageni_.NetFSD_upGrad_B4_Python\WEEK_4\Day_3_handson_Csharp\P5_Shopping_cart\P5_Shoping_cart\bin\Debug\net10.0\P5_Shopping_cart.exe (process 18188) exited with code 0 (0x0).
Press any key to close this window . . .
```

Level-2 Problem 5: Vehicle Rental System

Scenario:

A vehicle rental company wants a system where different vehicle types calculate rental charges differently.

Requirements:

1. Create a base class Vehicle with properties Brand and RentalRatePerDay.
2. Create derived classes Car and Bike.
3. Override CalculateRental(int days) method.
4. Car adds insurance charge of 500 per rental.
5. Bike offers 5% discount on total rental.

Technical Constraints:

- Use encapsulation with proper access modifiers.
- Apply runtime polymorphism.
- Validate number of rental days.

Expectations:

- Use base class reference to call overridden methods.
- Implement clean class hierarchy.
- Display final rental cost.

Learning Outcome:

- Master inheritance and polymorphism.
- Implement real-world OOP scenarios.
- Improve object-oriented design skills.

Sample Input:

Car RentalRatePerDay = 2000, Days = 3

Sample Output:

Total Rental = 6500

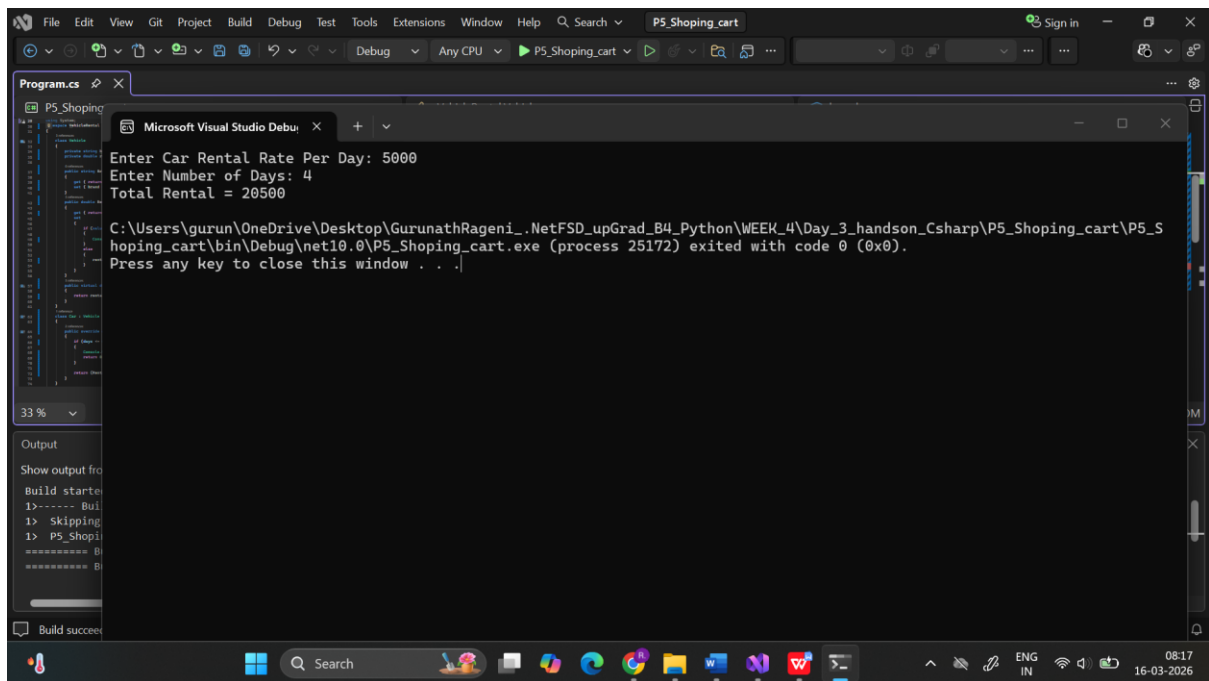
Code Screenshot:

```
Program.cs*  

C# P5_Shopping_cart

29 using System;
30 namespace VehicleRental
31 {
32     3 references:
33     class Vehicle
34     {
35         private string brand;
36         private double rentalRatePerDay;
37
38         0 references:
39         public string Brand
40         {
41             get { return brand; }
42             set { brand = value; }
43         }
44
45         3 references:
46         public double RentalRatePerDay
47         {
48             get { return rentalRatePerDay; }
49             set
50             {
51                 if (value < 0)
52                 {
53                     Console.WriteLine("Invalid rental rate");
54                 }
55                 else
56                 {
57                     rentalRatePerDay = value;
58                 }
59             }
60         }
61
62         3 references:
63         public virtual double CalculateRental(int days)
64         {
65             return rentalRatePerDay * days;
66         }
67     }
68
69     1 reference:
70     class Car : Vehicle
71     {
72         2 references:
73         public override double CalculateRental(int days)
74         {
75             if (days <= 0)
76             {
77                 Console.WriteLine("Invalid rental days");
78                 return 0;
79             }
80
81             return (RentalRatePerDay * days) + 500; // insurance charge
82         }
83     }
84
85     // Derived Class Bike
86     0 references:
87     class Bike : Vehicle
88     {
89         2 references:
90         public override double CalculateRental(int days)
91         {
92             if (days <= 0)
93             {
94                 Console.WriteLine("Invalid rental days");
95                 return 0;
96             }
97
98             double total = RentalRatePerDay * days;
99             return total - (total * 0.05); // 5% discount
100         }
101     }
102
103     0 references:
104     class Program
105     {
106         0 references:
107         static void Main(string[] args)
108         {
109             Console.Write("Enter Car Rental Rate Per Day: ");
110             double rate = Convert.ToDouble(Console.ReadLine());
111
112             Console.Write("Enter Number of Days: ");
113             int days = Convert.ToInt32(Console.ReadLine());
114
115             // Runtime Polymorphism
116             Vehicle vehicle = new Car();
117             vehicle.RentalRatePerDay = rate;
118
119             Console.WriteLine("Total Rental = " + vehicle.CalculateRental(days));
120         }
121     }
122 }
```

Output Screenshot:



```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P5_Shopping_cart Sign in
Microsoft Visual Studio Debug: P5_Shopping_cart Any CPU P5_Shopping_cart
P5_Shopping
Enter Car Rental Rate Per Day: 5000
Enter Number of Days: 4
Total Rental = 20500
C:\Users\gurun\OneDrive\Desktop\GurunathRageni_.NetFSD_upGrad_B4_Python\WEEK_4\Day_3_handson_Csharp\P5_Shopping_cart\P5_S
hoping_cart\bin\Debug\net10.0\P5_Shopping_cart.exe (process 25172) exited with code 0 (0x0).
Press any key to close this window . . .
33 %
Output
Show output from
Build started
1>----- Build
1> Skipping
1> P5_Shopping
Build succeeded
```